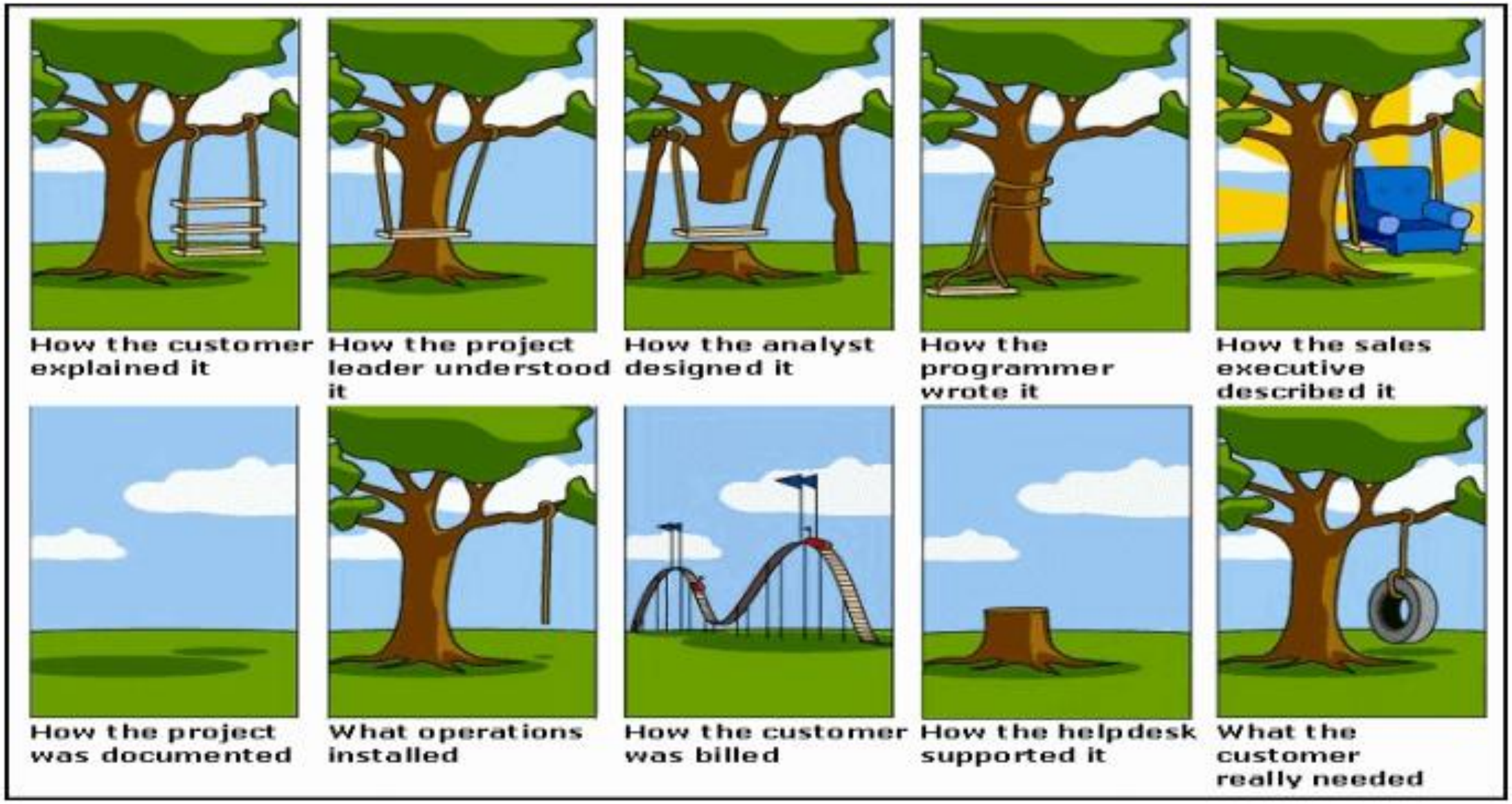
Chapter 3: Agile Software Development

Dr. Fatma ElSayed

Dr. Ahmed Yousry

The Problem(s) with Software Development



The Problem(s) with Software Development

- A Customers often **do not know exactly what they want**
- Not all requirements can be identified at **the beginning**.
- Requirements and business priorities **change** throughout the project.
- Changes often lead **to costly rework** in plan-driven methods.
- Rapid development and delivery: **A basic requirement**.

How can we build software that adapts to continuous change?

Traditional *(Plan-Driven)* Software Development

- Assumes that requirements can be **defined early** and **remain stable**.
- Requires **extensive planning** and **documentation** before development begins.
- Changes during development are often **expensive** and **time-consuming**.
- Customer **feedback** usually comes **late** in the project.
- Long development cycles **delay software delivery** and business value.

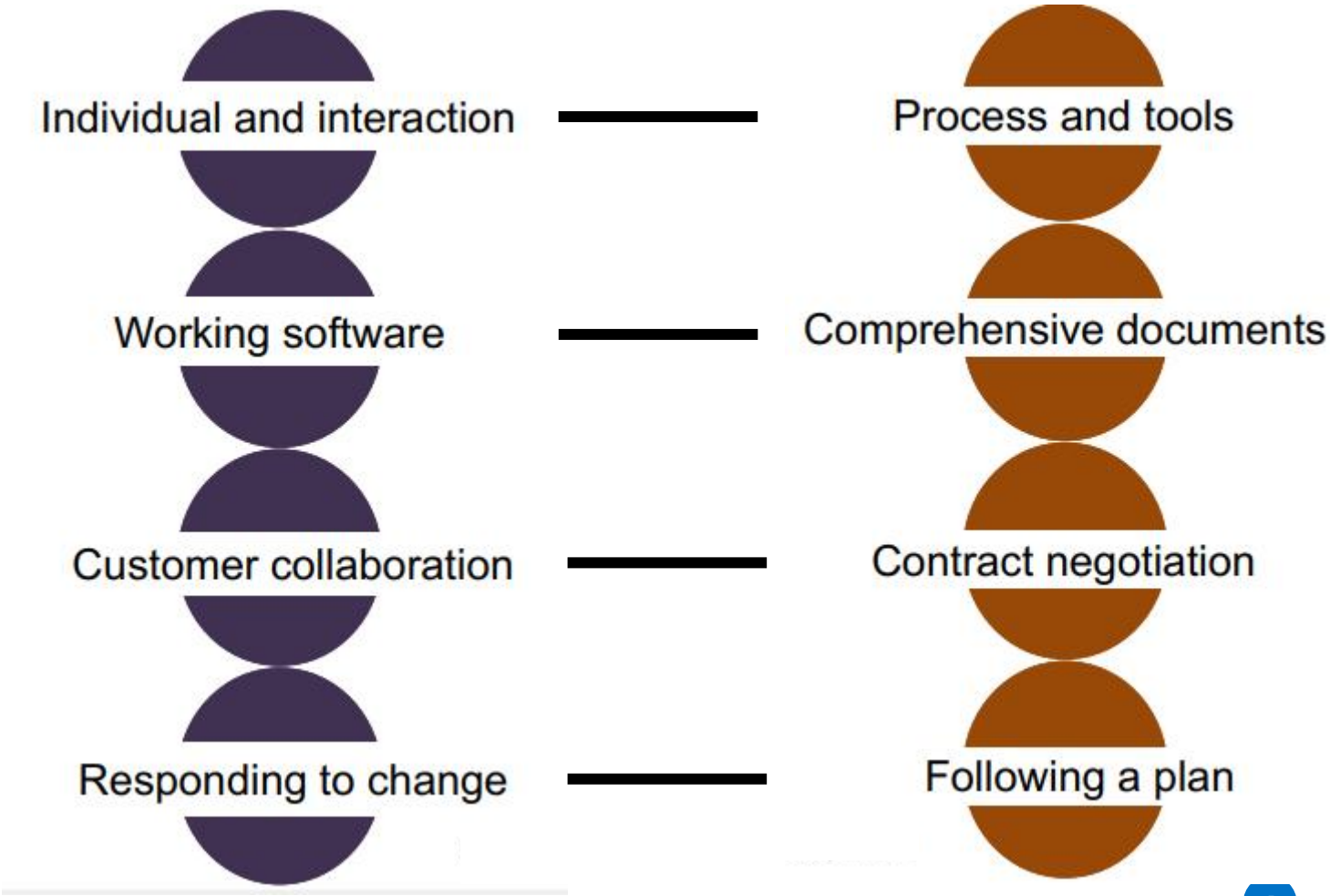
Agile was **not created because plan-driven methods are "bad."** It was created because **software development itself is unpredictable.**

What to do? “Agile” is born

“Agile Manifesto”

We are uncovering better ways of developing software by doing it and helping others do it.

Through this work we have come to value:



Key Agile Principles

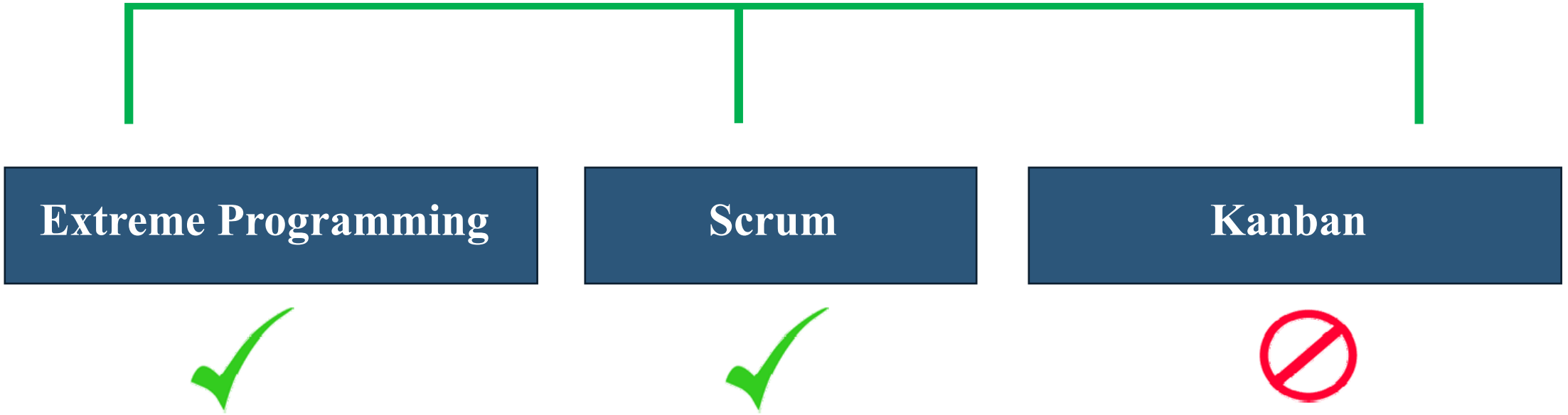




Key Agile Principles

- **Customer involvement:** customers should be closely involved throughout the development process.
- **Incremental delivery:** the software is developed in increments based on customer's priorities
- **People, not process:** let the development team use their skills and choose how they work.
- **Embrace change: e**xpect the system requirements to change and so design the system to accommodate these changes.
- **Maintain simplicity:** focus on simplicity in both the software being developed and in the development process.

Agile Development Techniques



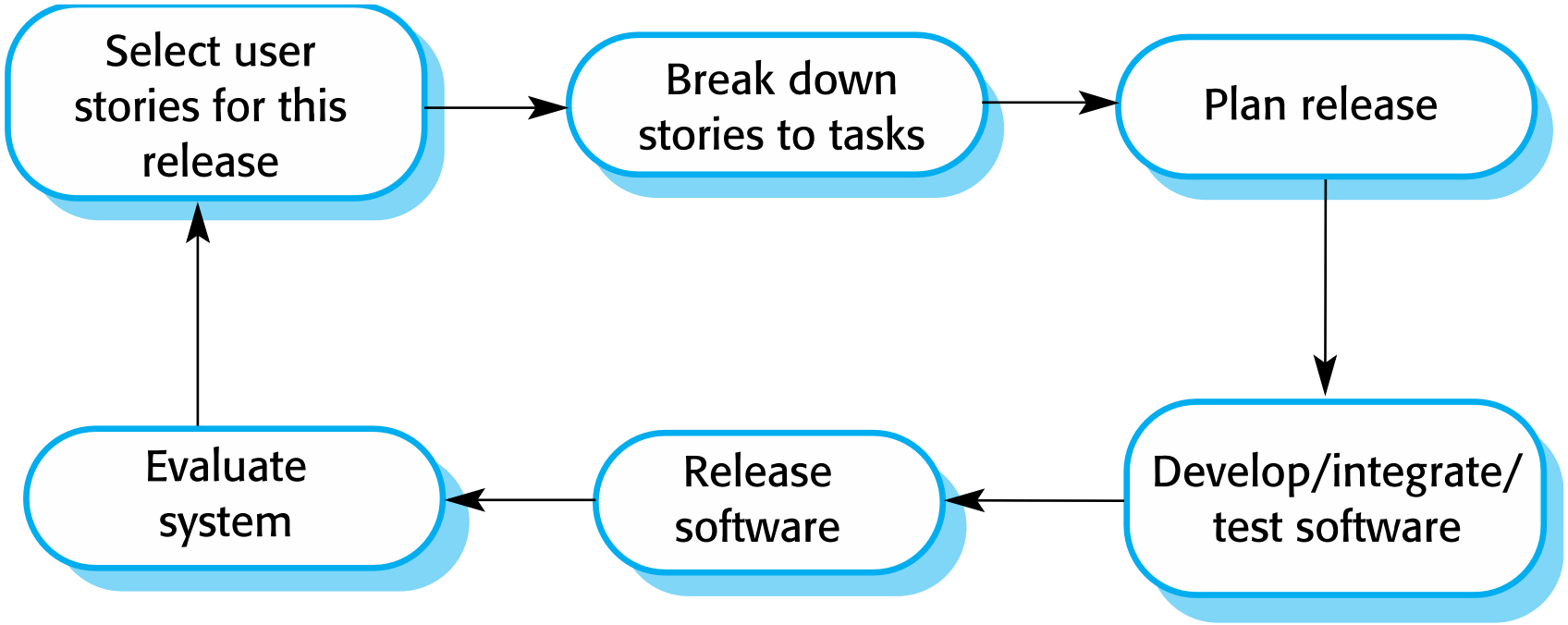
…… And many others

Extreme Programming

Extreme Programming (XP)

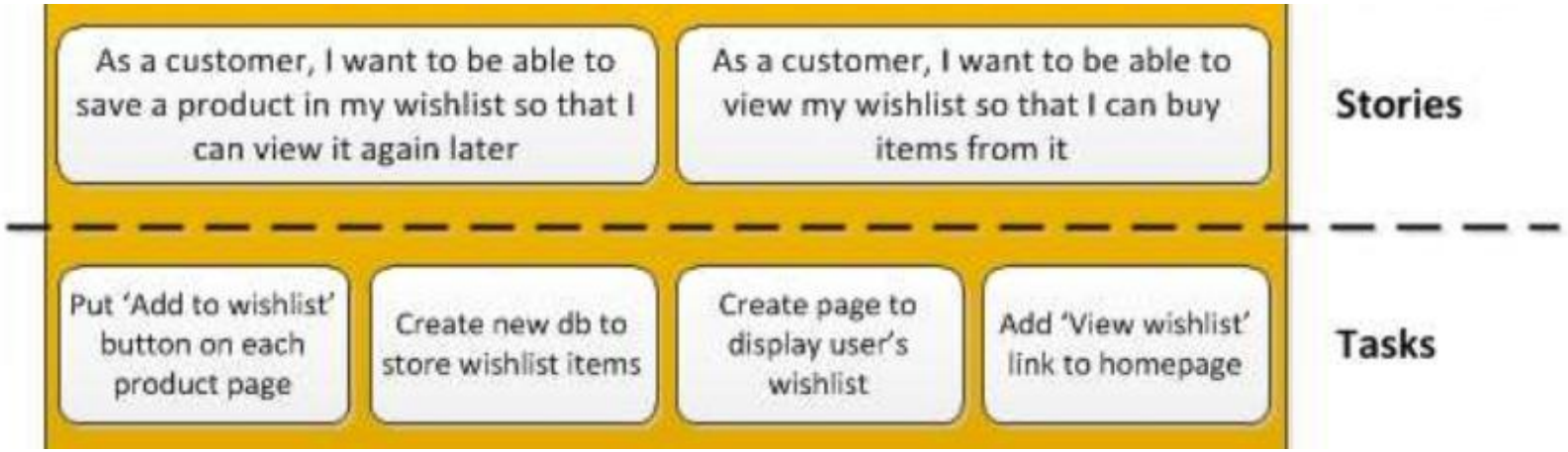
- **Extreme programming (XP)** is perhaps the best known and most widely used of the agile methods, developed in the late 1990s.
- Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.
  - New versions may be built several times per day;
  - Increments are delivered to customers every 2 weeks;
  - All tests must be run for every build and the build is only accepted if tests run successfully.

The Extreme Programming Release Cycle



Stories

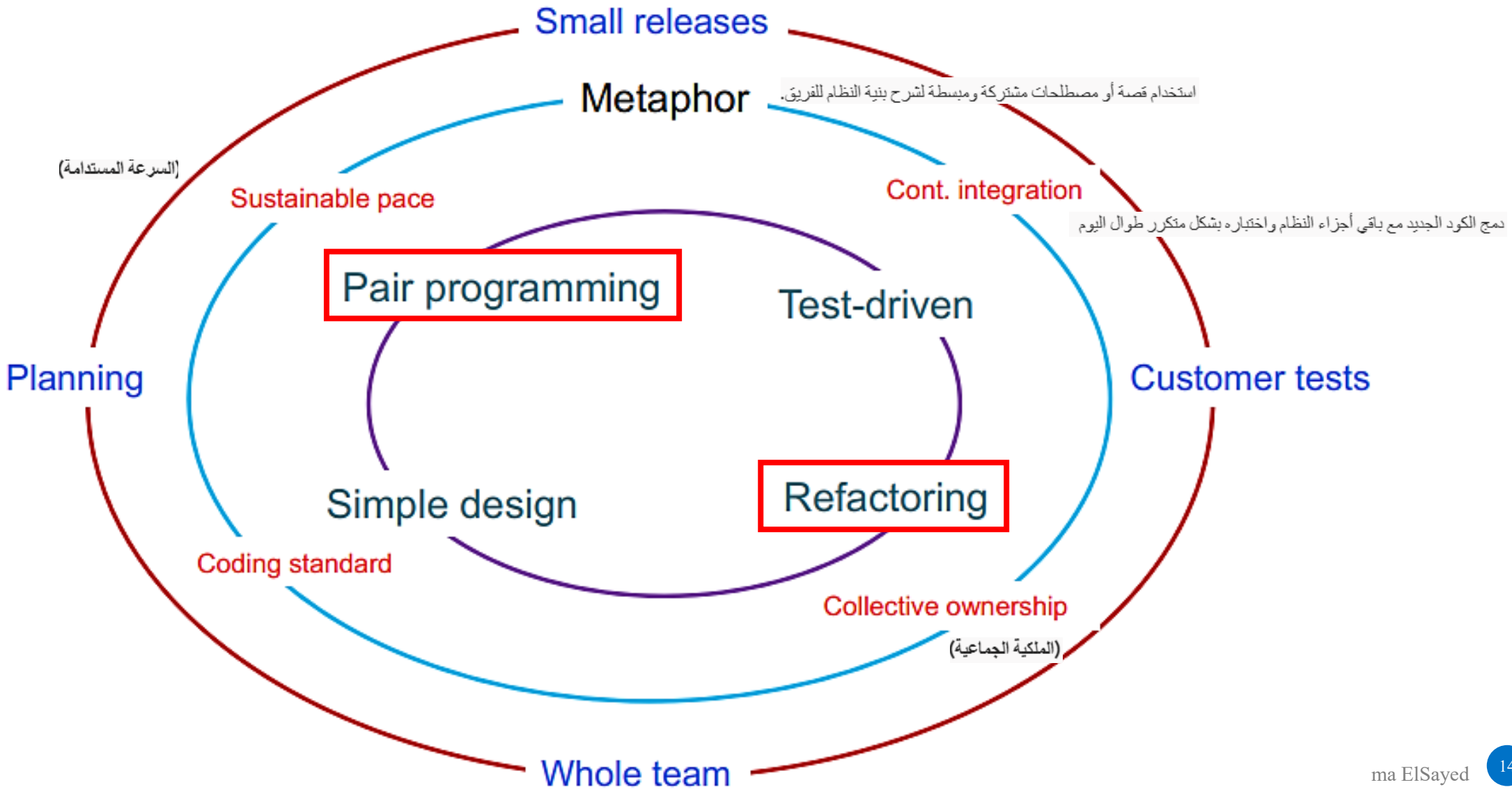
- A story is an individual feature or requirement that the client *(and business)* wants. It is something that is deliverable.
- Stories are often written in a **specific** format:
  - As a *[end user of the required feature]*
  - I want *[actual thing the user wants to be able to do once the feature is live – so often contains a verb]*
  - So that *[why they want this feature / the benefit this feature brings]*



User Stories for Requirements

- In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- User requirements are expressed as user stories or scenarios.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

Extreme Programming Practices



Refactoring

Refactoring

- A general problem with incremental development is that it tends to degrade the software structure, so changes to the software become harder and harder to implement.
  - Essentially, developers solve problems as they arise, which can lead to **duplicated code**, **inappropriate reuse of software components**, and gradual degradation of the overall system structure.
- Extreme programming tackles this problem by suggesting that the software should be constantly refactored.

Refactoring is the practice of improving the internal structure of existing code without changing its external behavior.



Refactoring cont.

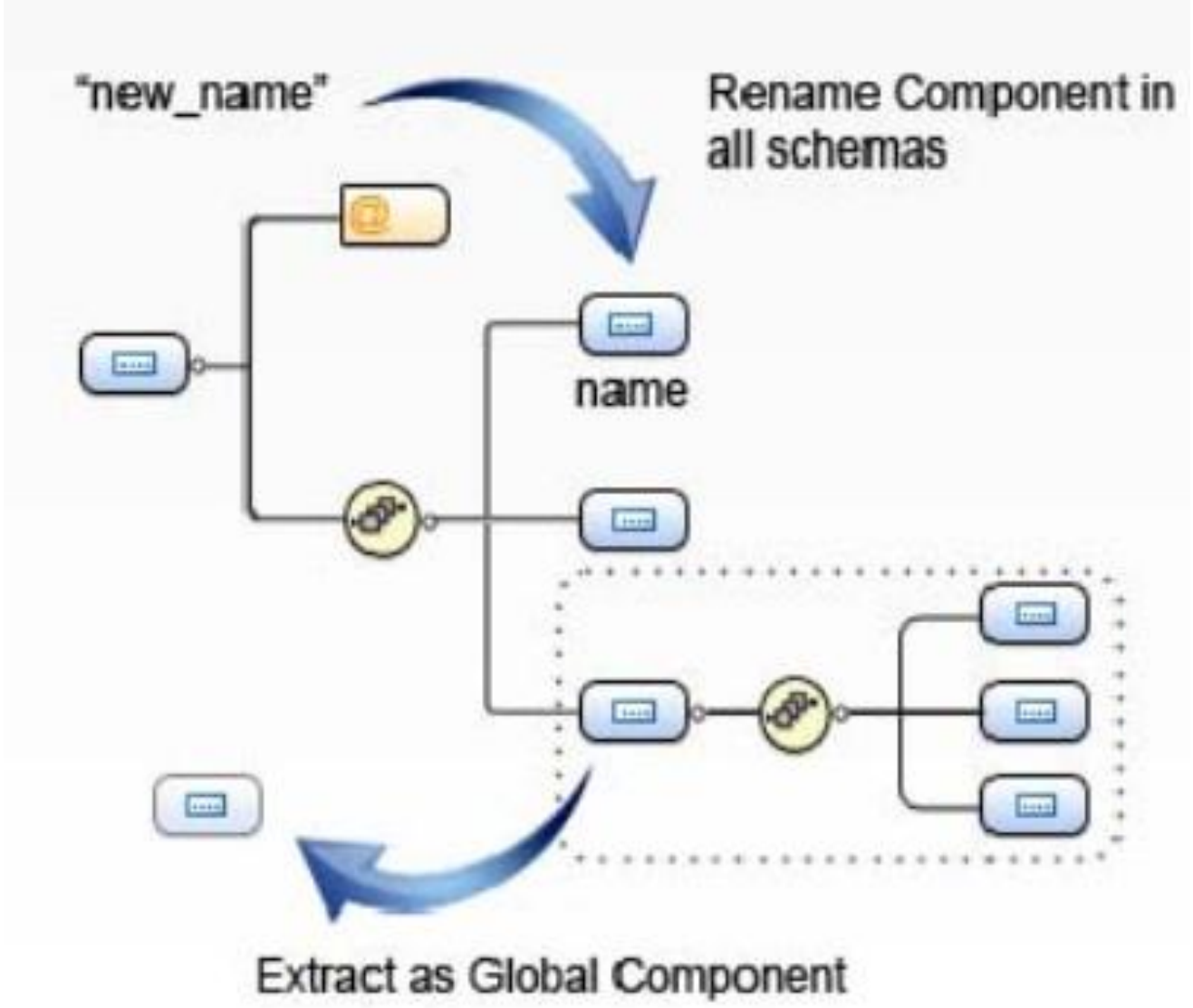
- This means that the programming team look for possible improvements to the software and implement them immediately.

Examples of refactoring include

- The **reorganization of a class** hierarchy to remove duplicate code,
- The **cleanup and renaming** of attributes and methods.
- The **replacement of code** with calls to methods defined in a program library.

This improves the understandability of the software and so reduces the need for documentation.

Refactoring cont.



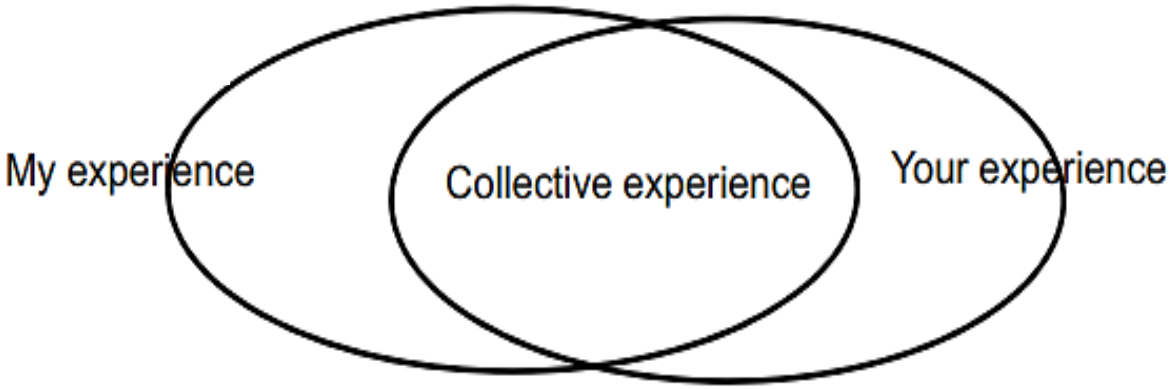
Pair Programming

Pair Programming

- Two people working together at a single computer.
- Two people working at a single computer will add as much functionality as two working separately **except** that it will be much higher in quality.



A review process



Pair Programming cont.

- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- The same pairs do not always program together. Rather, pairs are created **dynamically** so that all team members work with each other during the development process.

It encourages **refactoring** as the whole team can benefit from improving the system code.

Pair Programming cont.

Review





Idea sharing

Dynamicity





Responsibility

Scrum

Scrum

- Scrum is **a general agile method** that **focuses** on **managing iterative development** rather than specific agile practices.

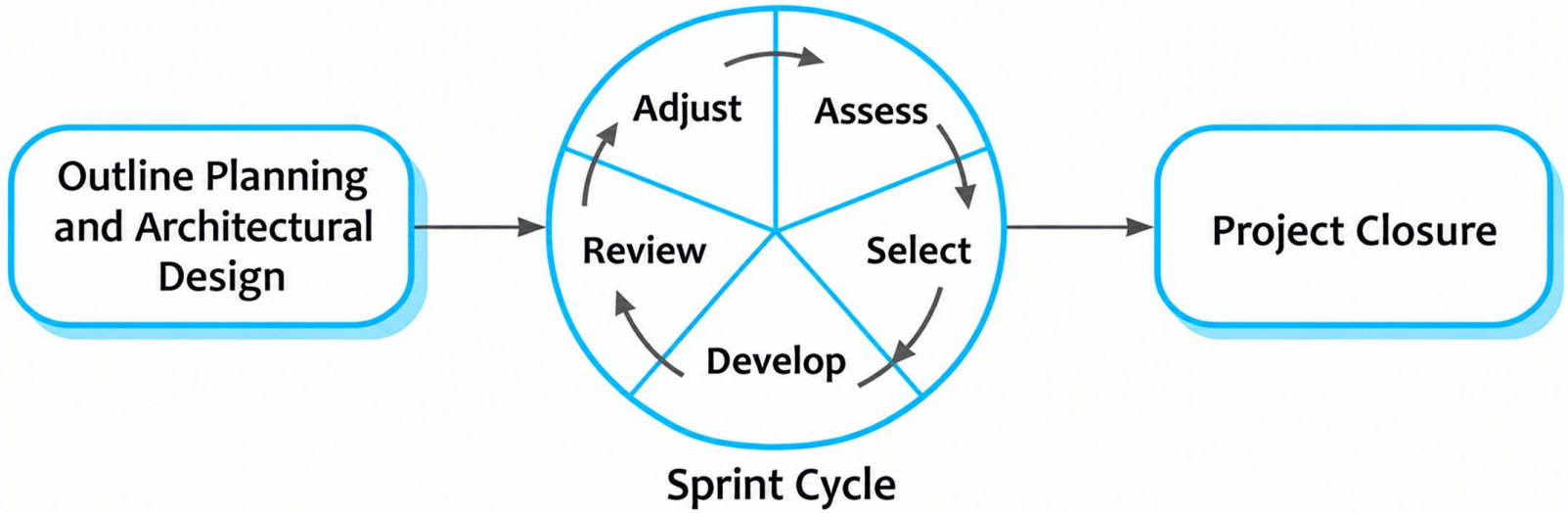
- Scrum **does not prescribe** the use of **programming practices** such as pair programming and test-first development. It can therefore be used with more **technical agile approaches**, such as XP, to provide **a management framework** for the project.



Scrum Process

There are three phases in Scrum

Finally, the **project closure** phase wraps up the project, completes required documentation such as user manuals, and assesses the lessons learned from the project.



Sprint Cycle

Length

- Normally 2-4 weeks
- Corresponds to a development iteration of a release

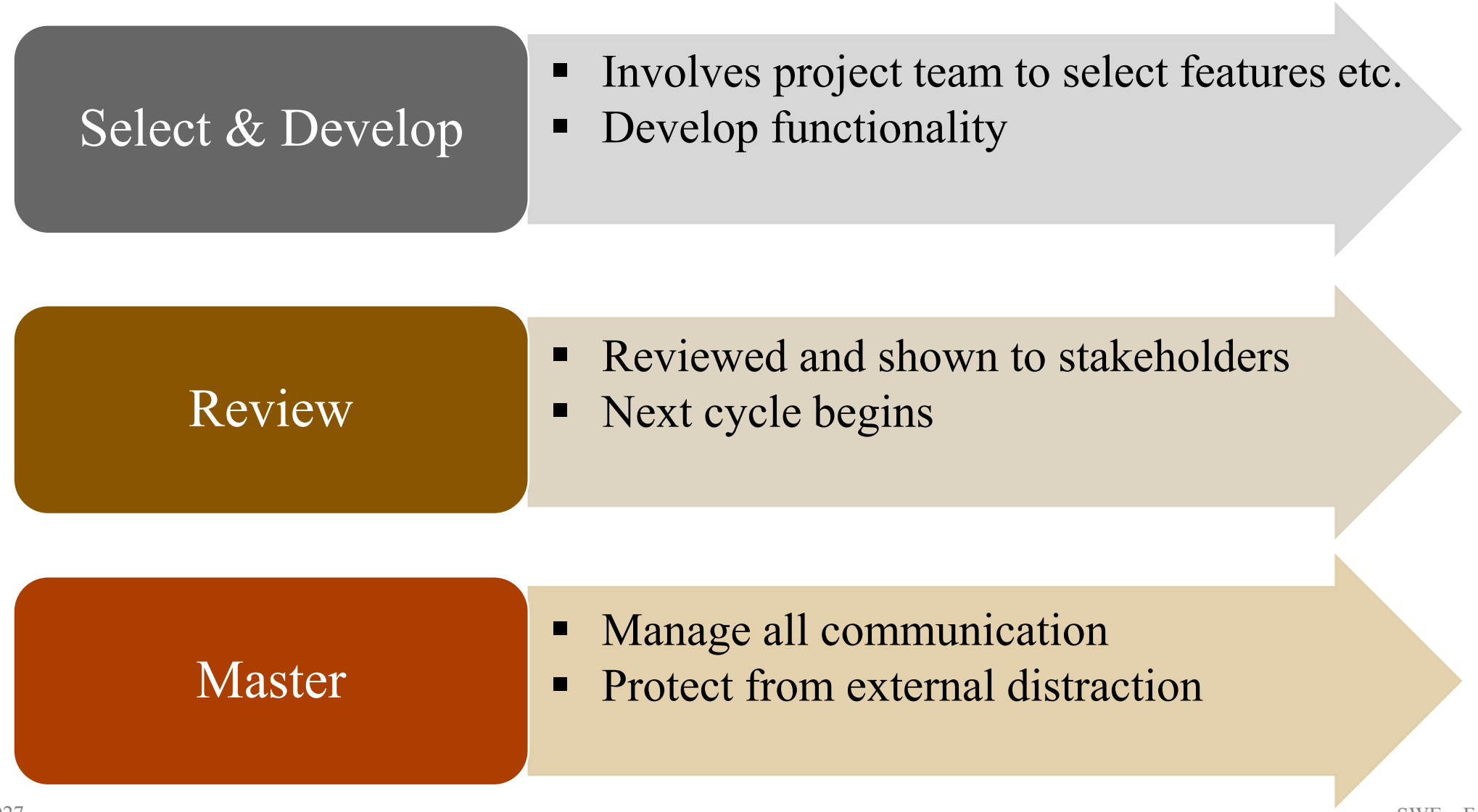
Adjust

- **Start point:** backlog of product
- **Backlog** is a list of work to be done on the project

Assess

- **Priorities** and risks are reviewed
- Customer is involved and can introduce new requirements

Sprint Cycle



Sprint Cycle: Explained

1. Sprints are fixed length, **normally 2–4** weeks. They correspond to the development **iteration** of a release.
2. The **starting point for planning** is the **product backlog**, which is the list of work to be done on the project.
    - During the *sprint assessment*, *priorities and risks* are reviewed. The customer is *closely involved and can introduce new requirements* or tasks at the start of each sprint.
3. The **selection phase** involves all of the project team who work with the **customer** to **select the features and functionality** to be developed during the sprint.

Sprint Cycle: Explained

4. Once these are agreed, the team **organizes themselves** to develop the software
   - During this stage the team is isolated from the customer and the organization, with all communications channeled through the so-called **'Scrum master'**
   - The role of the Scrum master is to protect the development team from **external distractions**.
5. At the end of the sprint, the work done is **reviewed** and presented to stakeholders.

   The next sprint cycle then begins.

Scrum Master

- The idea behind Scrum is that the **whole team** should be empowered to make decisions, so the term **‘project manager’** has been intentionally avoided.

Rather, the ‘Scrum master’ is a facilitator who

- Arranges daily meetings
- Tracks the backlog of work to be done
- Records decisions
- Measures progress against the backlog
- Communicates with customers and management outside of the team

Scrum Master

- The idea behind Scrum is that the **whole team** should be empowered to make decisions, so the term **‘project manager’** has been intentionally avoided.

Rather, the ‘Scrum master’ is a facilitator who

- Arranges daily meetings
- Tracks the backlog of work to be done
- Records decisions
- Measures progress against the backlog
- Communicates with customers and management outside of the team

Thank You